

CSE 585: Inference Lecture Series Oct. 7, 2025

Arnav Shah, Roe Tsimhoni, Rohit Saripalle, Wonbin Jin



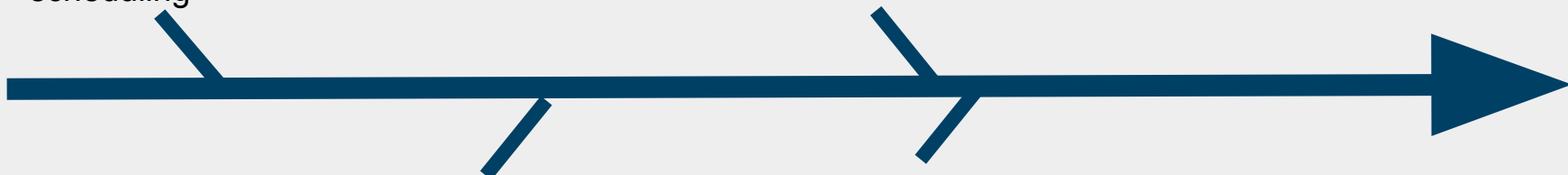
Timeline

Orca (OSDI'22):
Iteration-level
scheduling

DistServe (OSDI'24):
Disaggregation of prefill
and decode

vLLM (SOSP'23):
Memory efficient KV
cache management

Sarathi-Serve (OSDI'24):
Chunked prefills and stall
free mixed batching

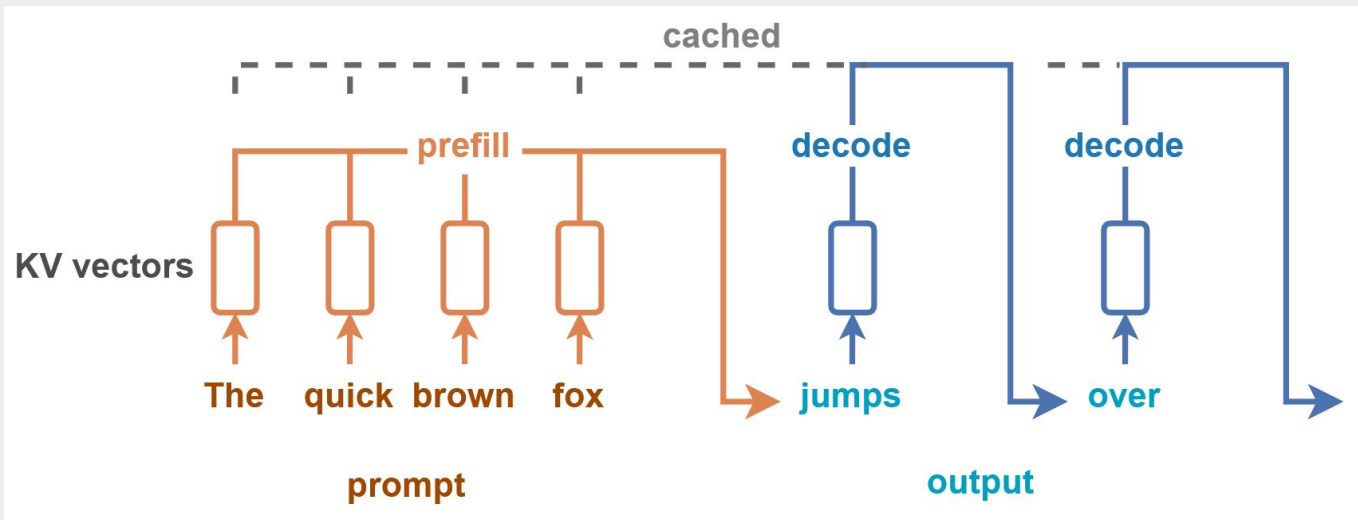




Inference Phases

Prefill Phase: Processing of the input prompt and produces the first output token

Decode Phase: Generates the rest of the output tokens one at a time



Inference Phases

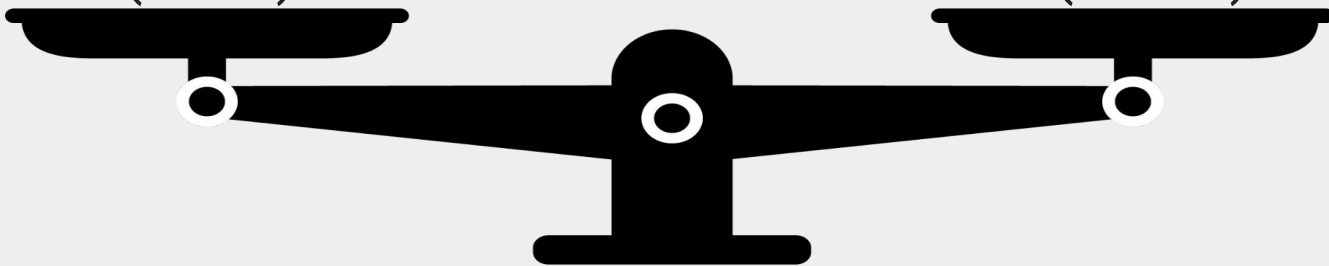
	Prefill	Decode
Compute characteristic	Compute Bound	Memory Bound
Tensor Parallelism	Effective at lower request rates	Reduce TPOT but with diminishing returns
Pipeline Parallelism	Effective at higher request rates	Scale throughput linearly
Batching	Applied selectively	Needed for high GPU utilization
Performance metric	TTFT target	TPOT/TBT target



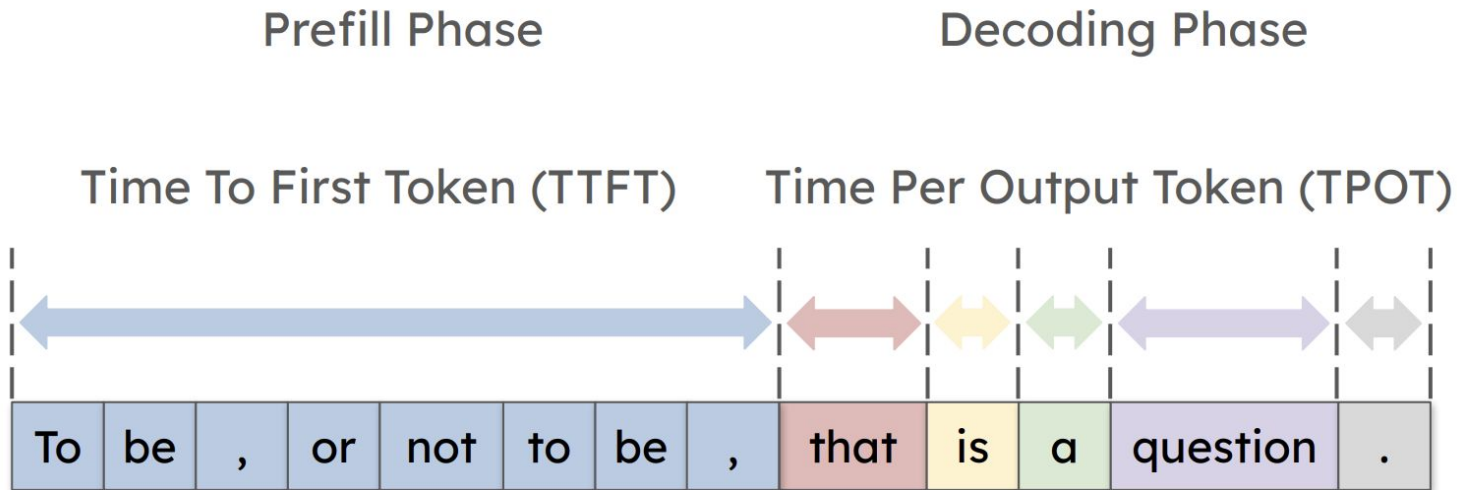
Prefill vs Decode

Prioritize prefill
for TTFT
(Orca)

Prioritize decode for
TPOT
(vLLM)



Service Level Objectives



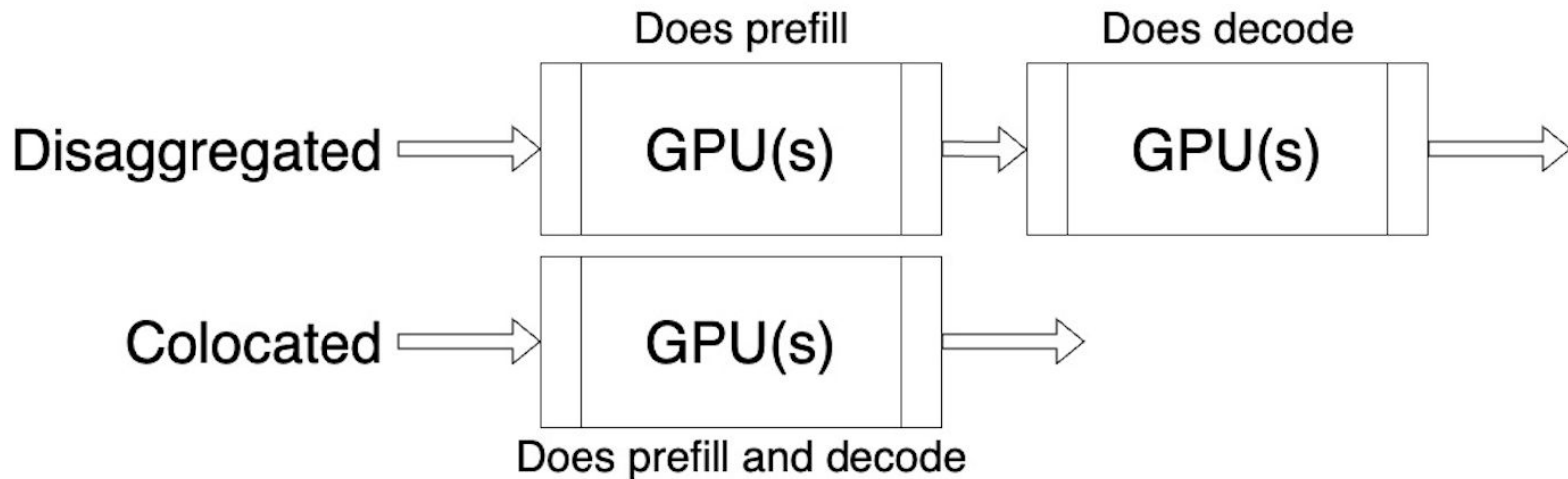
Related Works

- Model Serving Systems
 - Traditional systems: Clipper, TensorFlow-Serving
 - LLM-focused systems: Orca, vLLM
 - Prefill-decode disaggregation: SplitWise, **DistServe**, TetriInfer
 - FastServe: preemptive scheduling to reduce head-of-line blocking
 - Sheng et al.: fairness in iteration-level batching
 - APIServe: adopts chunked-prefills for multi-turn API serving
- GPU & Model Optimizations
 - GPU efficiency: FasterTransformers, CocoNet Model-level, **Sarathi-Serve**
 - Model Optimizations: multi-query attention, quantization, and mixture-of-experts

DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving



Disaggregating Prefill and Decoding



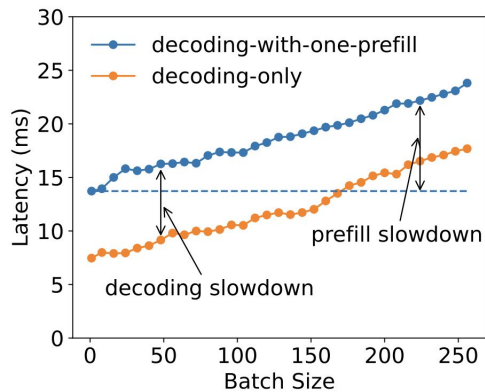


Goodput-Optimized

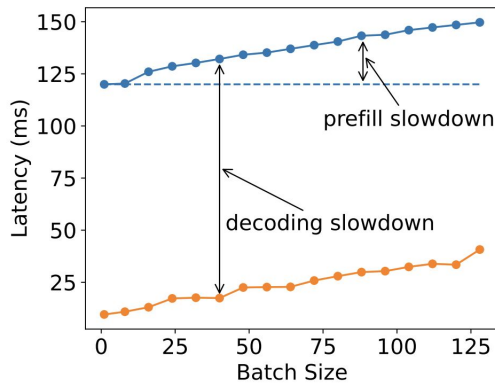
	TTFT	TPOT
Chat	Low ($< 1s$)	Match read speed (~ 100ms)
Search	Very Low (~ 200ms)	Match read speed (~ 100ms)
Program	Very Low (~ 200ms)	Very Low (~ 50ms)

Goodput
=
Request
throughput that
adhere to SLOs

Why Disaggregate?



(a) Input length = 128



(b) Input length = 1024

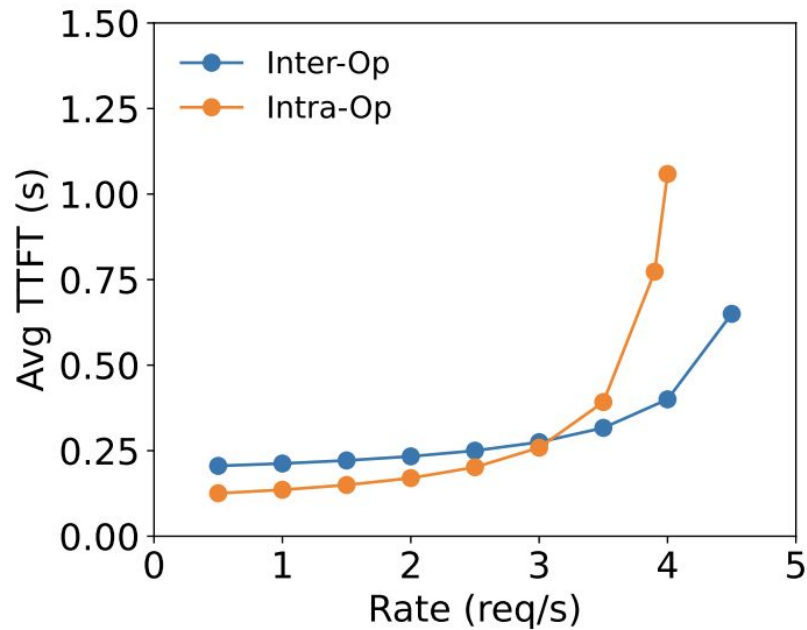
Figure 2: Batch execution time when serving a 13B LLM as batch size increases. Compared between a decoding-only batch and the batch adding one more prefill job.

Prefill Decode Interference

Prefill

- Even a single prefill job can saturate GPU compute
 - Batching just adds more latency
- At high request rates
 - Pipeline parallelism needed
- Most cases
 - Tensor parallelism is preferred

Prefill



Decode

- Single decode job doesn't saturate GPU compute
 - Batching necessary for high GPU utilization
- For strict TPOT requirements
 - Tensor parallelism may be needed
- Most cases
 - Pipeline parallelism can scale throughput almost linearly with GPU

Decode

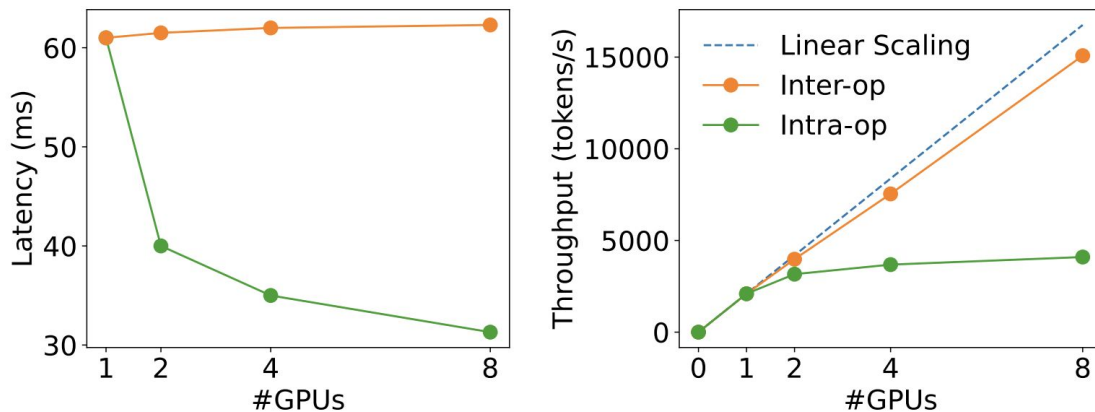
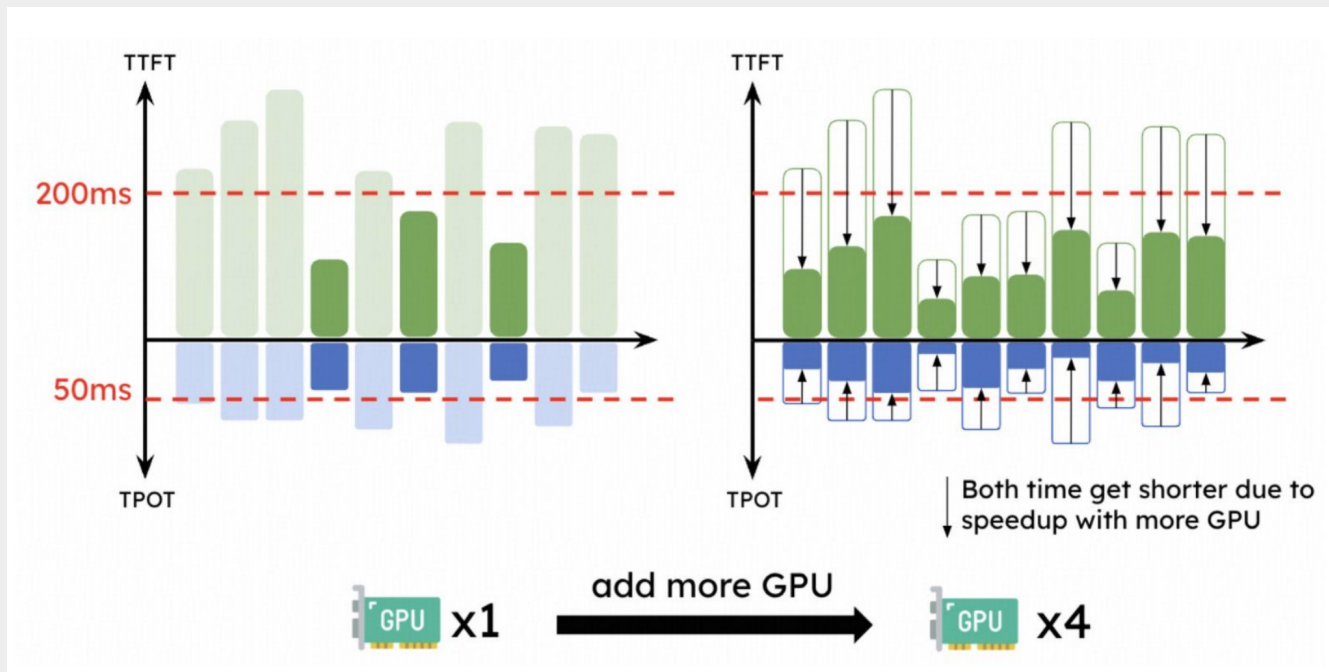


Figure 5: Decoding phase latency and throughput when serving a 13B LLM with batch size = 128 and input length = 256 under different parallel degrees.

Why Disaggregate?

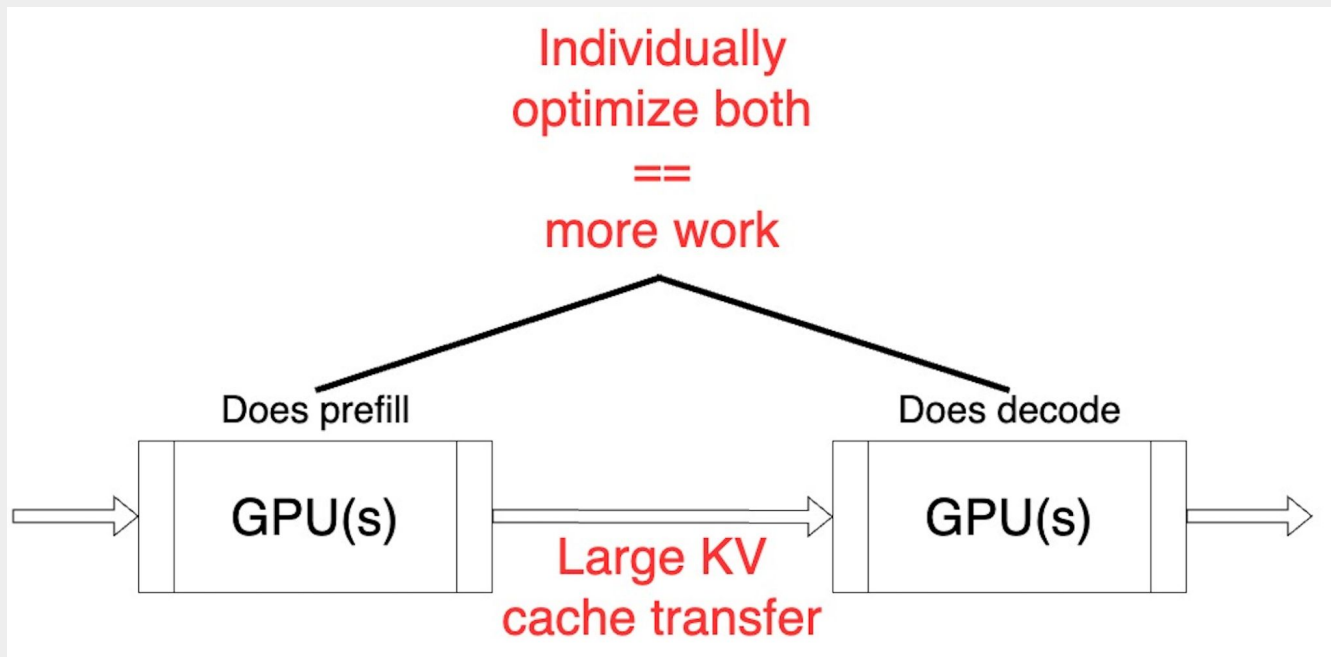


Why Disaggregate?

- No interference → Optimize each phase separately
- Scale each phase separately → 5 prefill instances to 1 decode instance
- Tradeoff raw throughput for goodput



Disaggregation Challenges



DistServe

- Cluster configuration for minimizing KV cache transfer overhead
- Automatic discovery of optimal parallelism strategy
- Online scheduling that adapts to workload in real time

Placement algorithm

Infiniband: 800 GB/s inter node communication.
Communication overhead negligible

High Node-Affinity: InfiniBand available

Low Node-Affinity: InfiniBand unavailable

Placement for High Node-Affinity Cluster

1. Optimize prefill and decode parallelism separately
2. Replicate to meet traffic rate
 - Problem: No closed form way to predict SLO attainment under bursty workloads
 - Solution?

Simulator

Model parameters:

- Hidden size
- Number of heads
- Head size

Input data characteristics:

- Batch size
- Token lengths

Arithmetic intensity of operation

- Compute or memory bound



Execution time of prefill and
decode phase

Simulator

$$T_{prefill} = \underbrace{C_1 \cdot (4th^2 + 2thm)}_{\text{Compute latency from GEMM}} + \underbrace{C_2 \cdot \frac{3ht^2}{b}}_{\text{Memory latency from FlashAttention}} + \underbrace{C_3}_{\text{System overheads}}$$

Simulator

$$T_{decode} = \underbrace{C_4 \cdot (4h^2 + 2hm)}_{\text{Memory latency from GEMM}} + \underbrace{C_5 \cdot 3ht}_{\text{Memory latency from FlashAttention}}$$

Evaluation: Simulator Accuracy

Rate (req/s)	vLLM		DistServe-Low	
	Real System	Simulator	Real System	Simulator
1.0	97.0%	96.8%	100.0%	100.0%
1.5	65.5%	65.1%	100.0%	100.0%
2.0	52.8%	51.0%	99.3%	99.3%
2.5	44.9%	46.1%	87.3%	88.3%
3.0	36.7%	38.3%	83.0%	84.1%
3.5	27.8%	28.0%	77.3%	77.0%
4.0	23.6%	24.1%	70.0%	68.9%

Table 2: Comparison of the SLO attainment reported by the simulator and the real system under different rates.



High Node Affinity:

Algorithm 1 High Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

$config_p, config_d \leftarrow \emptyset, \emptyset$

for $intra_op \in \{1, 2, \dots, M\}$ **do**
 for $inter_op \in \{1, 2, \dots, \frac{N \times M}{intra_op}\}$ **do**

Enumerate all possible
parallelism configurations

if $\frac{G.size}{inter_op \times intra_op} < C$ **then**

$config \leftarrow (inter_op, intra_op)$

$\hat{G} \leftarrow \text{parallel}(G, config)$

$config.goodput \leftarrow \text{simu_prefill}(\hat{G}, W)$

if $\frac{config_p.goodput}{config_p.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**

$config_p \leftarrow config$

Estimate prefill
goodput and take
best one

$config.goodput \leftarrow \text{simu_decode}(\hat{G}, W)$

if $\frac{config_d.goodput}{config_d.num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**

$config_d \leftarrow config$

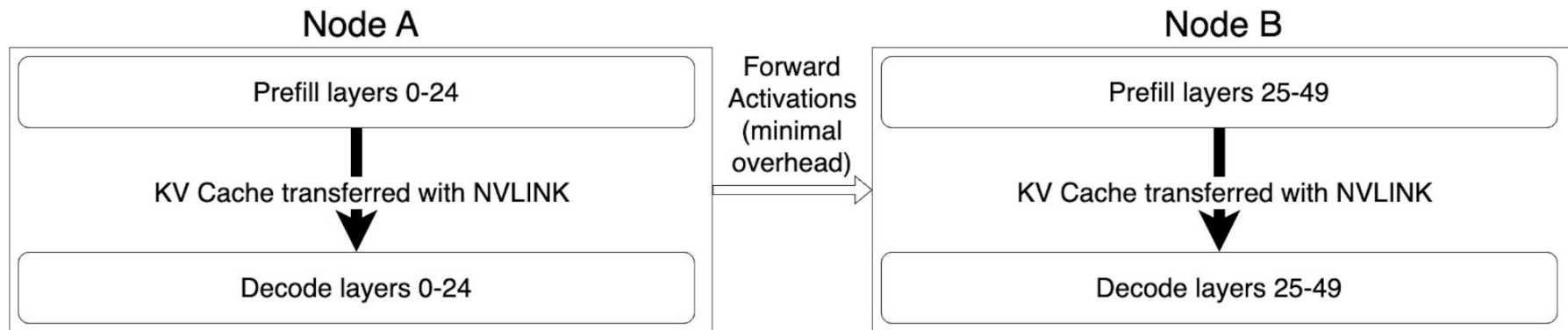
Estimate decode
goodput and take
best one

$n, m \leftarrow \lceil \frac{R}{config_p.goodput} \rceil, \lceil \frac{R}{config_d.goodput} \rceil$

$best_plm \leftarrow (n, config_p, m, config_d)$

return $best_plm$

Low Node Affinity: Can't fit full prefill and decode instances in one node for big models





Low Node Affinity:

Algorithm 2 Low Node-Affinity Placement Algorithm

Input: LLM G , #node limit per-instance N , #GPU per-node M , GPU memory capacity C , workload W , traffic rate R .

Output: the placement $best_plm$.

$config^* \leftarrow \emptyset$

for $inter_op \in \{1, 2, \dots, N\}$ **do**

Enumerate possible PP
stage breakdowns

$\mathcal{P} \leftarrow \text{get_intra_node_configs}(G, M, C, inter_op)$

Generate possible TP
configurations for a stage

for $P_p \in \mathcal{P}$ **do**

for $P_d \in \mathcal{P}$ **do**

if $P_p.num_gpus + P_d.num_gpus \leq M$ **then**

$config \leftarrow (inter_op, P_p, P_d)$

$\hat{G}_p, \hat{G}_d \leftarrow \text{parallel}(G, config)$

$config.goodput \leftarrow \text{simulate}(\hat{G}_p, \hat{G}_d, W)$

if $\frac{config.*goodput}{config.*num_gpus} < \frac{config.goodput}{config.num_gpus}$ **then**

$config^* \leftarrow config$

Estimate goodput and
take best one

$n \leftarrow \lceil \frac{R}{config.*goodput} \rceil$

$best_plm \leftarrow (n, config^*)$

return $best_plm$

Online Scheduling

- Reduce pipeline bubbles
- Combat burstiness
- Replan configuration

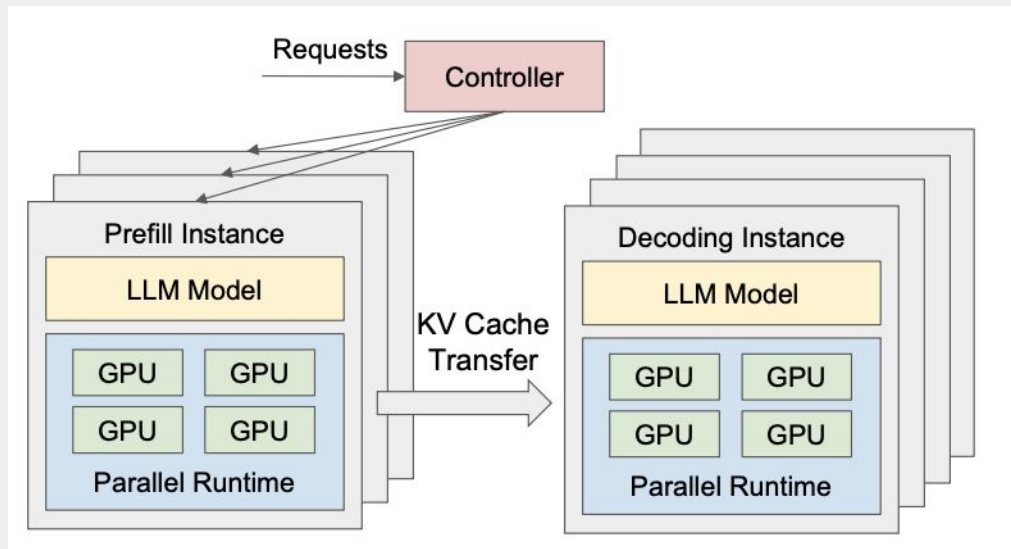
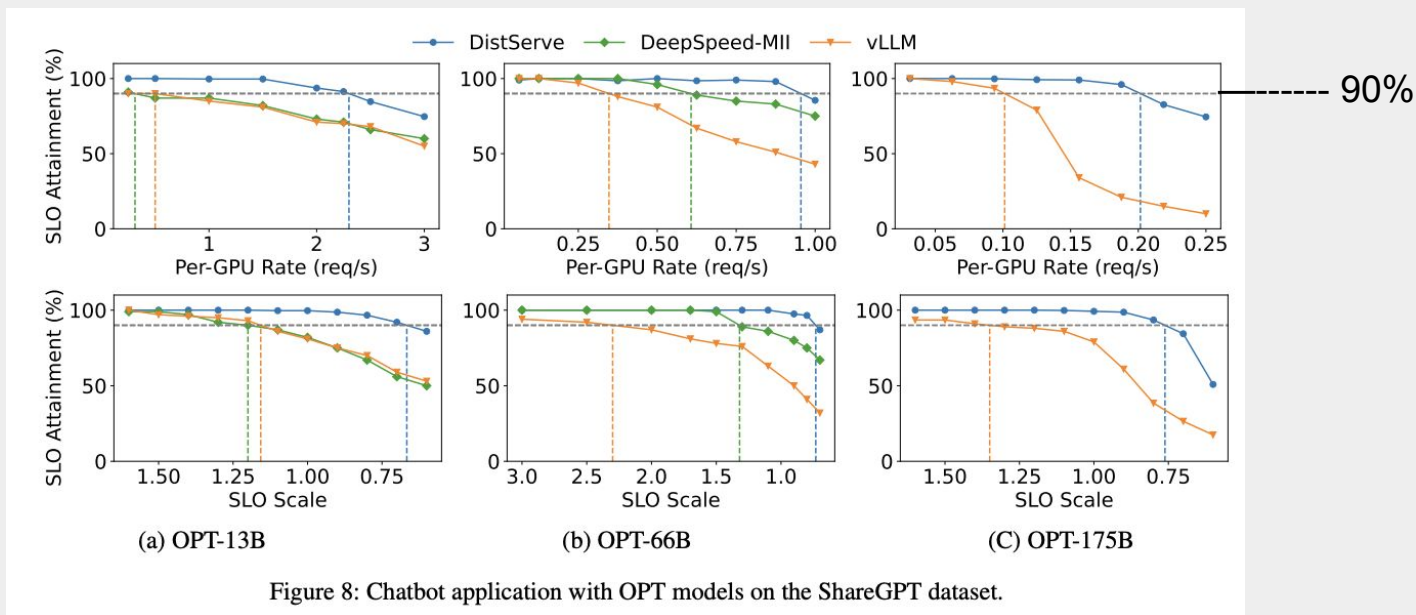


Figure 6: DistServe Runtime System Architecture

Evaluation: Experiment Setup

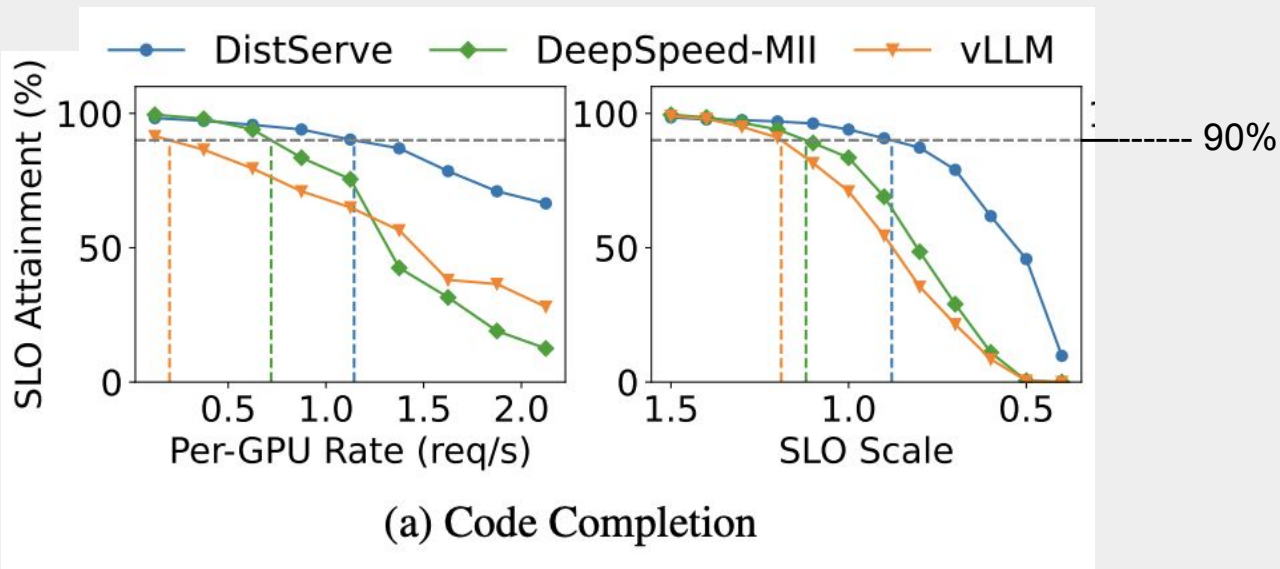
- Cluster testbed
- Model and workload setup
 - Chatbot
 - Code Completion
 - Summarization
- Metrics
- Baselines
 - vLLM
 - DeepSpeed-MII

Evaluation: Chatbot



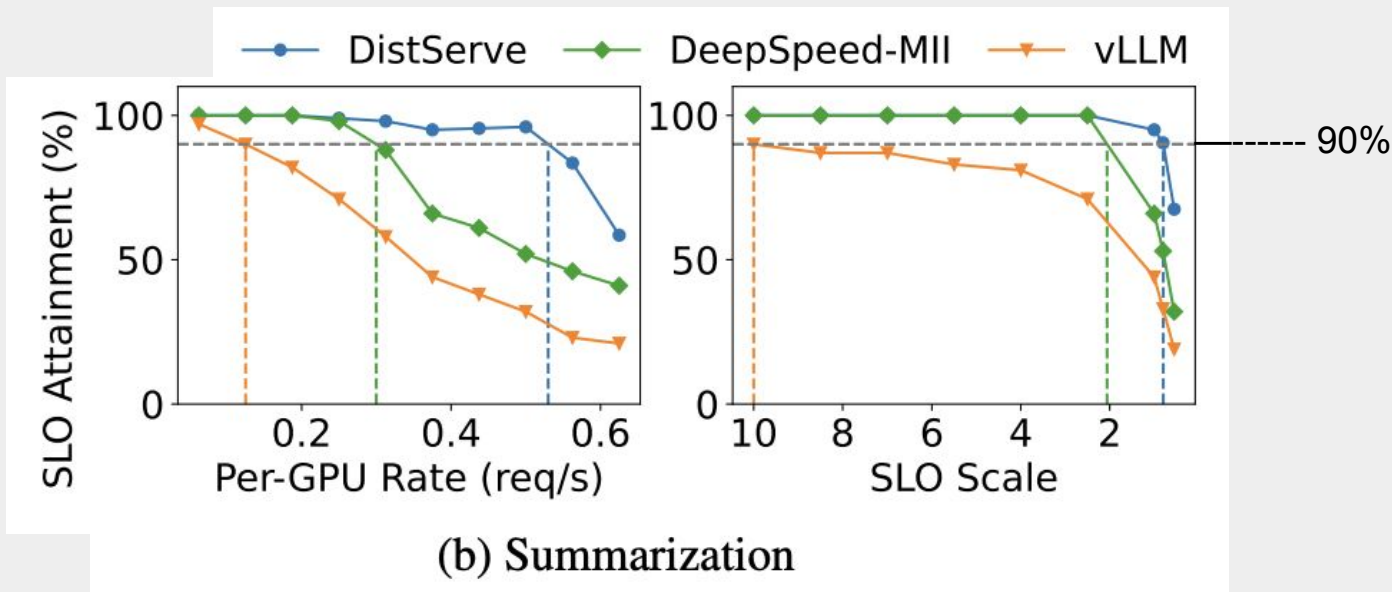


Evaluation: Code Completion





Evaluation: Summarization



Evaluation: Algorithm Running Time

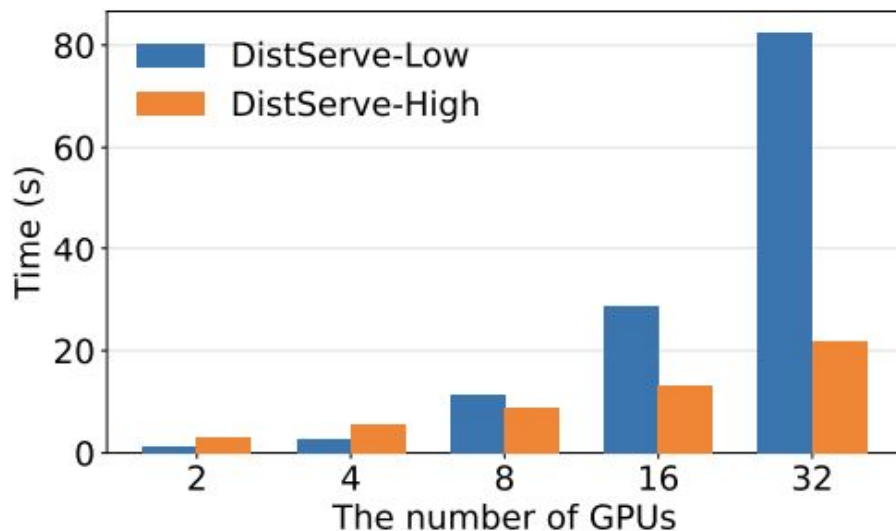


Figure 12: Algorithm Running Time

Critique

Strengths:

- Detailed analysis of the problem and weaknesses of existing solutions
- Simple but effective idea that provides another dimension of tradeoffs
- Thorough evaluation and convincing results

Weaknesses:

- Doesn't work with one GPU
- Doesn't address fault tolerance and cascading failures

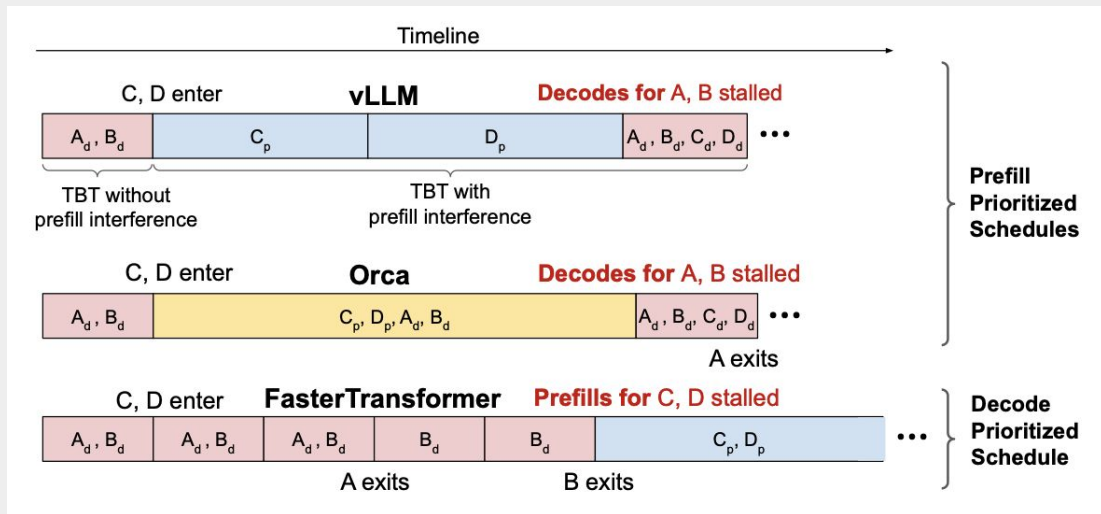
Future Work

- Fault tolerance
- Runtime policies
- Long-context scenarios

Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve

Inference Challenges

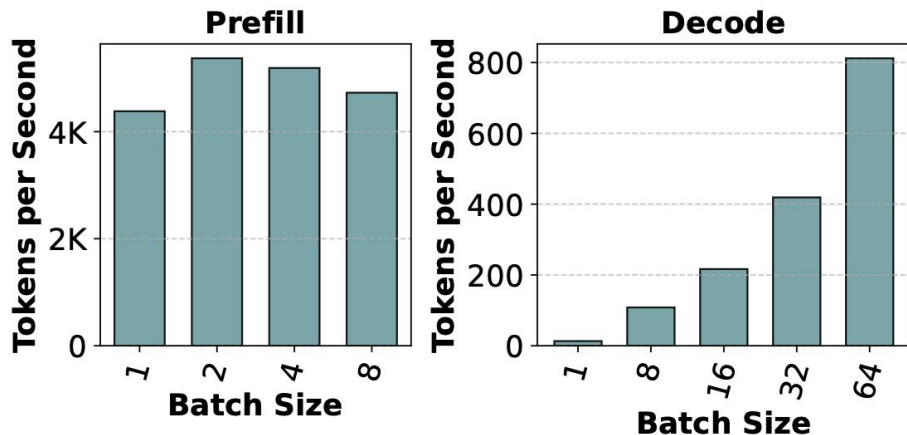
- Prioritizing prefills optimizes throughput but sacrifices TBT (generation stalls)
- Prioritizing decodes optimizes TBT but sacrifices throughput





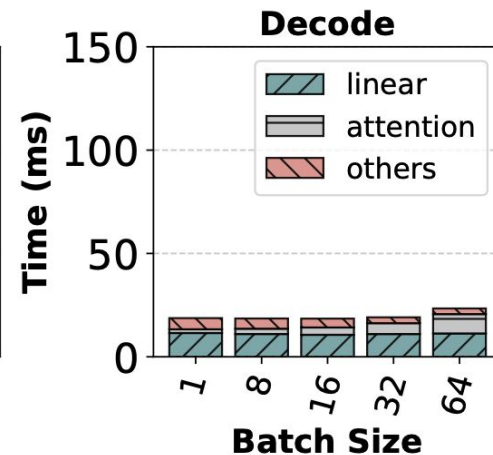
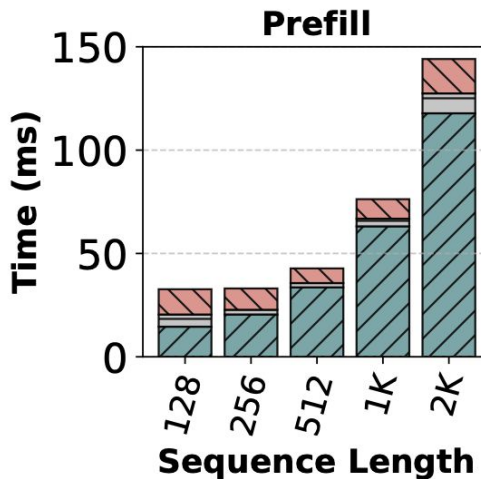
Observations

- Throughput of prefill and decode phases for Mistral-7B running on 1 A100 GPU
- Prefill saturates with single request
- Decode scales linearly with batch size

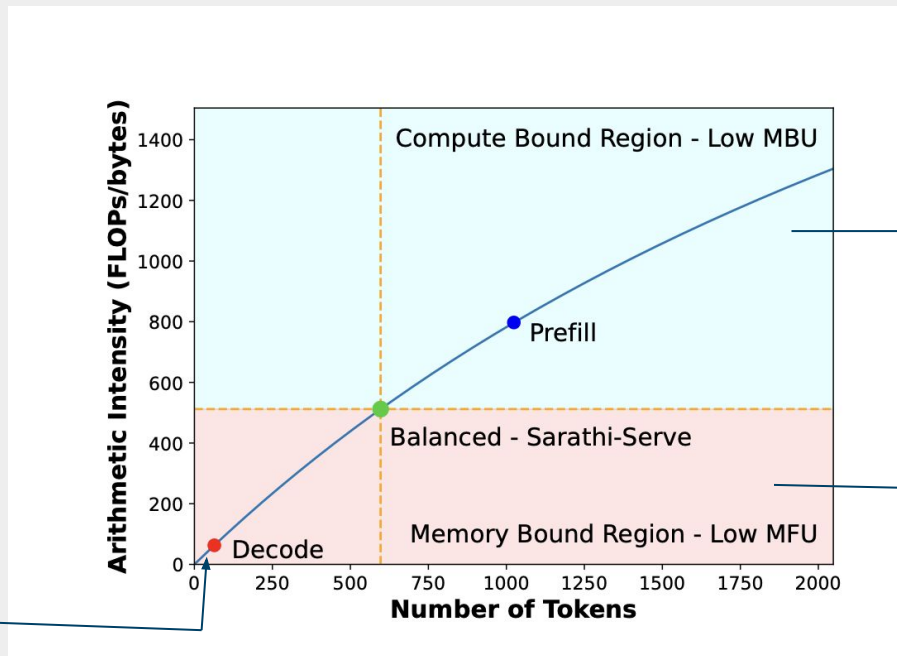


Observations Cont.

- Prefill and decode time with different input sizes for Mistral-7B running on single A100 GPU
- Linear layers contribute to majority of time in both phases



Observations Cont.



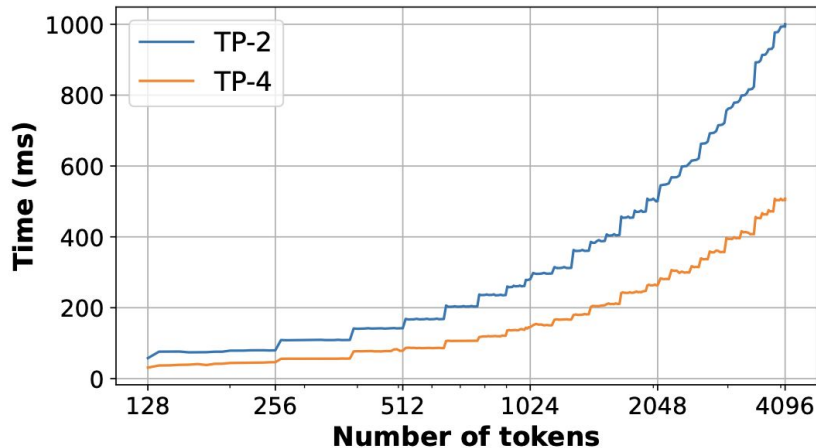
Constrained due to
memory overhead in
decode phase

Latency dominated
by compute
Grows linearly with
num tokens

Latency dominated
by weight fetch time
Independent of num
tokens

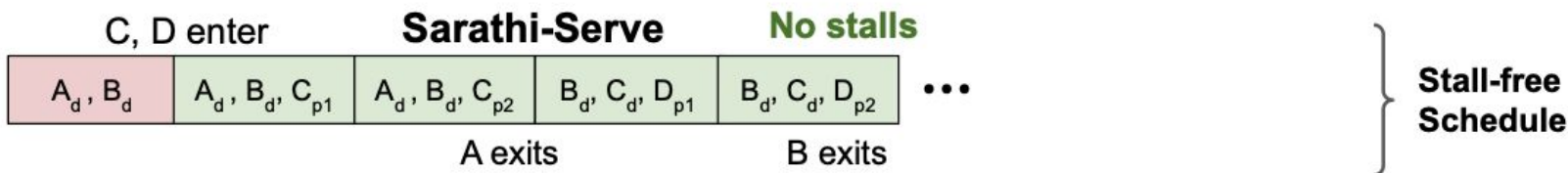
Observations Cont.

- Linear layer execution time as function of number of tokens in a batch for LLaMA2-70B on A100s with different tensor parallel degrees
- Once number of tokens in a batch crosses threshold it become compute bound



Approach

- Chunked prefills: split large prefills into smaller chunks – just enough to consume the leftover compute budget in decode batches
- Stall free schedule: No generations stalls



How it Works

- Step 1: Always admit all ongoing decodes into the batch.
- Step 2: Add partially completed prefills (chunked).
- Step 3: Admit new requests if token budget allows.

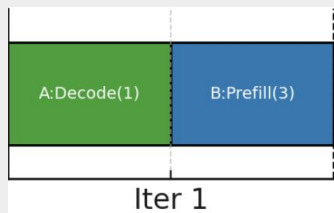
Example

- 2 requests:
 - A - in decode phase
 - Will generate 3 tokens
 - B - in prefill phase
 - Prefill is 6 tokens, decode will generate 2
 - Token budget = 4

Iteration 1

- A in decode -> contributes one token
- B prefill size
 - Assume chunk size is 3

	Prefill tokens	Decode tokens
Request A	-	3
Request B	6	2

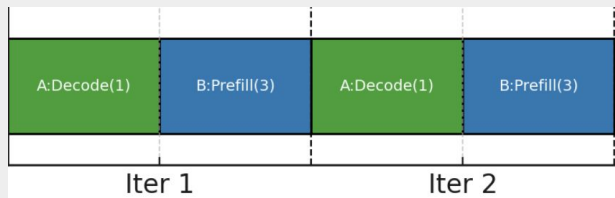




Iteration 2

- A still in decode -> 1 token
- B finishes prefill -> 3 tokens

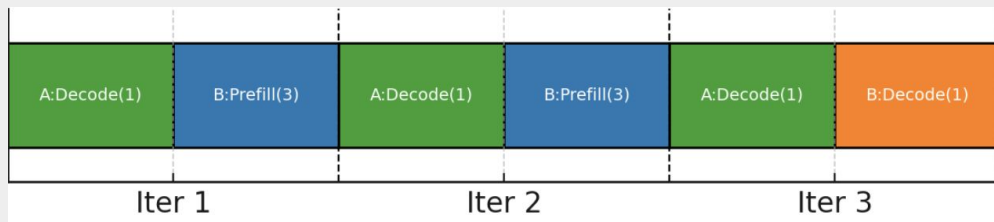
	Prefill tokens	Decode tokens
Request A	-	3
Request B	6	2



Iteration 3

- A finishes decode -> 1 token
- B begins decode -> 1 token

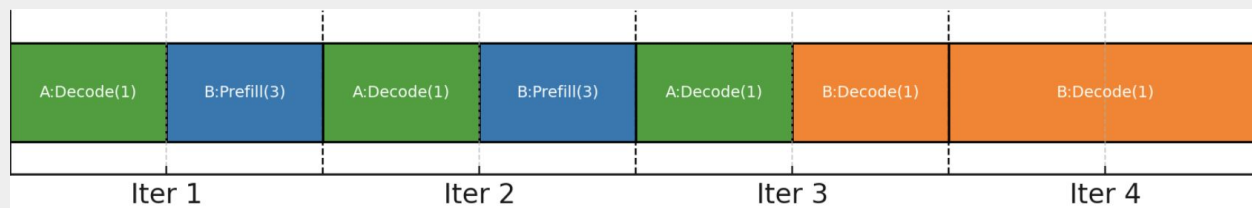
	Prefill tokens	Decode tokens
Request A	-	3
Request B	6	2



Iteration 4

- B finishes decode -> 1 token

	Prefill tokens	Decode tokens
Request A	-	3
Request B	6	2



Token Budget

- Based on TBT SLO requirement and chunked-prefills overhead
- One time profiling (Vidur) to determine token budget
- Takes into account
 - TBT SLO (latency bound)
 - Pipeline configuration (avoid bubbles)
 - GPU tile dimensions (avoid tile-quantization inefficiency)



Implementation

- Implemented on vLLM
- Added paged chunk prefill using:
 - FlashAttention v2
 - FlashInfer kernels
- Extended vLLM to support:
 - Multiple scheduling policies
 - Chunked-prefills
 - Pipeline parallelism
 - Telemetry system
- Communication: NCCL (tensor + pipeline parallel).

Evaluation

- What is the maximum load a model replica can serve under specific Service Level Objective (SLO) constraints with different inference serving systems and how does this load vary with varying SLO constraints?
- How does Sarathi-Serve perform under various deployments such as TP and PP?
- What is the overhead of chunked-prefills?
- What is the effect of each of chunked-prefills and stall-free batching in isolation as opposed to using them in tandem?

Evaluation

- Models:
 - Mistral-7B
 - Yi-34B
 - LLaMA2-70B
 - Falcon-180B
- Datasets:
 - openchat_sharegpt4
 - Arxiv_summarization
- Metrics:
 - TTFT (Median)
 - TBT (P99)

Model	Attention Mechanism	GPU Configuration	Memory Total (per-GPU)
Mistral-7B	GQA-SW	1 A100	80GB (80GB)
Yi-34B	GQA	2 A100s (TP2)	160GB (80GB)
LLaMA2-70B	GQA	8 A40s (TP4-PP2)	384GB (48GB)
Falcon-180B	GQA	4 A100s×2 nodes (TP4-PP2)	640GB (80GB)

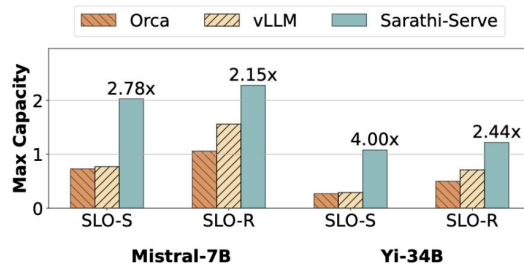
Table 1: Models and GPU configurations (GQA: grouped-query attention, SW: sliding window).

Dataset	Prompt Tokens			Output Tokens		
	Median	P90	Std.	Median	P90	Std.
<i>openchat_sharegpt4</i>	1730	5696	2088	415	834	101
<i>arxiv_summarization</i>	7059	12985	3638	208	371	265

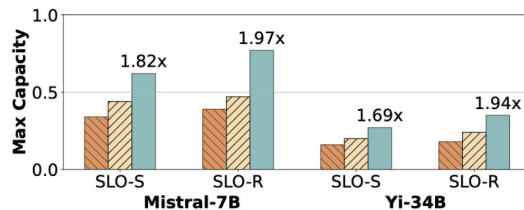
Table 2: Datasets used for evaluation.

Capacity Evaluation

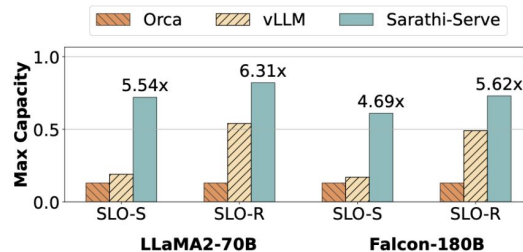
- 4-6x gains over Orca and vLLM



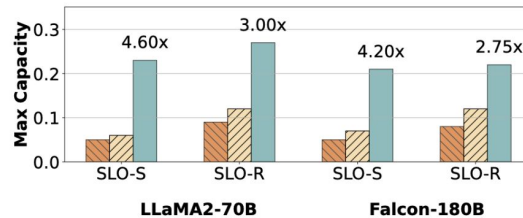
(a) Dataset: *openchat_sharegpt4*.



(b) Dataset: *arxiv_summarization*.



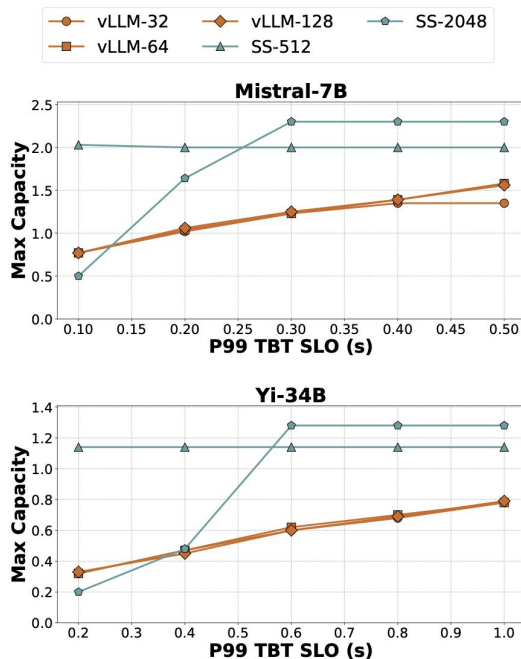
(a) Dataset: *openchat_sharegpt4*.



(b) Dataset: *arxiv_summarization*.

Throughput-Latency TradeOff

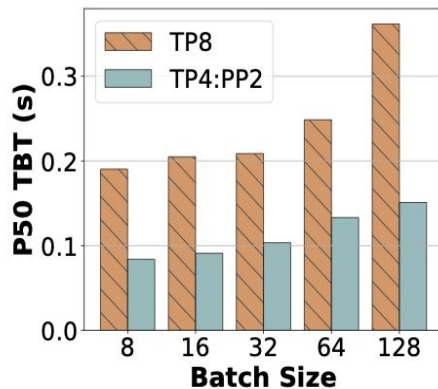
- Achieves 3.5× higher capacity than vLLM under strict SLO (100ms, Mistral-7B)
- Achieves 1.65× higher capacity than vLLM under relaxed SLO (1s, Yi-34B)



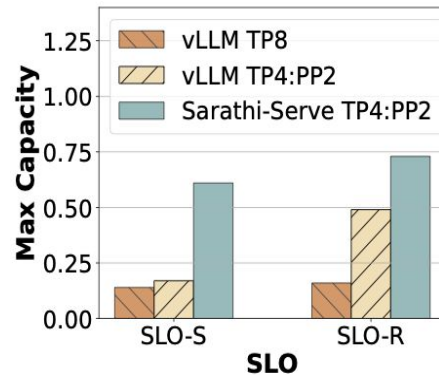


Pipeline Parallelism

- Sarathi-Serve TP-4 + PP-2:
 - Chunked-prefills smooth execution time across microbatches
 - Reduces pipeline bubbles
 - Achieves 1.48× higher capacity under relaxed SLO



(a) TBT (Falcon-180B).

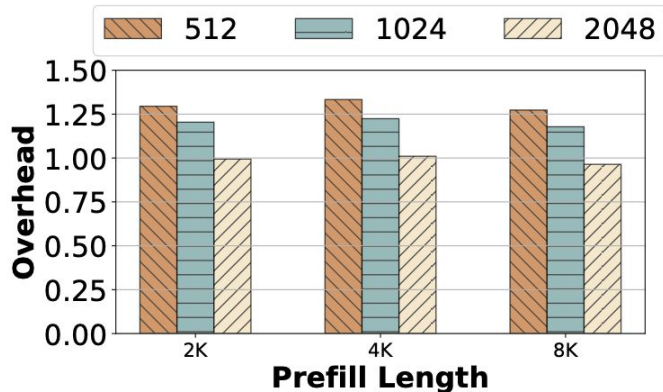


(b) Capacity (Falcon-180B).



Ablation Studies

- Overhead of chunked-prefills
- Analyzing the impact of hybrid-batching and chunking on latency



Scheduler	openchat_sharegpt4		arxiv_summarization	
	P50 TTFT	P99 TBT	P50 TTFT	P99 TBT
hybrid-batching-only	0.53	0.68	3.78	1.38
chunked-prefills-only	1.04	0.17	5.38	0.20
Sarathi-Serve (combined)	0.76	0.14	3.90	0.17

Critique

Strengths:

- Stall-Free Batching: Efficiently schedules prefills without delaying decodes
- Flexible Token Budget: Controls latency-throughput tradeoff for different SLOs
- Pipeline Parallelism Optimized: Chunked-prefills reduce pipeline bubbles, improving capacity

Weaknesses:

- Token Budget Profiling Required: profiling needed
- Implementation Complexity: Scheduler and chunked-prefill logic add overhead

Future Work

- Dynamic Token Budgeting: Automatically adapt token budgets based on workload characteristics and SLOs for better efficiency
- Broader Hardware & Network Evaluation: Test on heterogeneous clusters and different GPU types

Recap

Feature	Sarathi-Serve	DistServe
Prefill & Decode	On the same GPU, uses chunked-prefills	Disaggregated across GPUs, prefill & decode separated
GPU Utilization	Keeps GPU busy by overlapping prefills with decodes	High utilization for both prefill and decode instances
Latency Control	Token budget allows tuning latency vs throughput	Separate prefill and decode tuning and scaling
KV-Cache Management	KV-cache stays on same GPU, extra overhead for chunks	Transfer negligible with InfiniBand and smart placement
Deployment Considerations	Simpler single-GPU setup	Benefits from high-bandwidth multi-node hardware